Autonomous Synthesis of Hard RTL Designs via Iterative Repair and Modular Decomposition

Anonymous Author(s)

Abstract

Large language models can generate short register-transfer level (RTL) fragments, but benchmark-level hardware synthesis requires executable correctness: simulation, protocol consistency, and golden-output agreement. ArchXBench exposes this gap through a reported hard-task frontier beginning at Level 4 under direct prompting. This paper evaluates whether verifier-grounded autonomous synthesis can cross that frontier without confusing model failures with benchmark-contract failures. Four conditions are compared on ArchXBench Levels 3–6: single-shot generation (C1), monolithic golden-feedback repair (C2g), decomposition-guided iterative repair (C4i), and testbench-localized decomposition (C4tl). C4i solves five Level-3 numerical kernels at 3/3 seeds where both C1 and C2g fail. C4tl solves all four core Level-4 designs (including 16-point FFT/IFFT) at 5/5 seeds. Golden-verified Level-5 and Level-6 solves are obtained on convolution, image-processing, and AES designs. Decomposition is not universally superior: C2g solves several Level-5 rows that decomposed methods do not. The central contribution is an executable-contract-aware synthesis methodology: original-benchmark solves, minimally repaired executable contracts, and held rows are separated before success is claimed. Code and artifacts are available.[1]

CCS Concepts

• **Hardware** → **Hardware description languages and compilation**; **Electronic design automation**; • **Software and its engineering** → *Automatic programming*.

Keywords

benchmark evaluation, CEGIS, hardware verification, LLM agents, RTL synthesis, Verilog generation

ACM Reference Format:

Anonymous Author(s). 2027. Autonomous Synthesis of Hard RTL Designs via Iterative Repair and Modular Decomposition. In *Proceedings of 32nd Asia and South Pacific Design Automation Conference (ASP-DAC '27).* ACM, New York, NY, USA, 6 pages.

1 Introduction

Autonomous code agents close executable loops: generate a candidate, run a checker, inspect failures, and repair. For software tasks, this loop has changed what counts as a meaningful benchmark result. For hardware design, the question is harder: can an autonomous agent synthesize RTL that survives simulation, protocol assumptions, and golden-output comparison?

RTL correctness depends on bit widths, signedness, clocking, reset protocols, pipeline latency, and streaming handshakes. Hard RTL synthesis therefore tests whether autonomous agents can handle low-level, state-dependent, timing-sensitive design, not just surface-level code generation. The severity is practical: an RTL generator that compiles but mishandles latency, signed arithmetic, or output encoding can pass superficial checks while producing unusable hardware.

This paper studies the problem on ArchXBench, a benchmark suite for complex RTL synthesis [8]. Unlike VerilogEval [4] and RTLLM [5], which target smaller designs, ArchXBench spans arithmetic kernels, FFTs, filters, image processing, matrix multiplication, and cryptography across six difficulty levels. The ArchXBench paper reports a direct-generation frontier beginning at Level 4: lower levels are broadly reachable, while hard Level-4 rows such as FFT/IFFT expose near-zero zero-shot pass@5 behavior. If an autonomous synthesis loop can solve rows beyond it, the evaluation question changes from "can an LLM emit a complete design in one shot?" to "can a verifier-grounded agent converge to correct RTL?"

The central question is: *can verifier-grounded autonomous synthesis solve hard RTL benchmark tasks that direct generation does not solve, while maintaining a clear distinction between model failures and benchmark-contract failures?* The answer is partially affirmative. C4i solves five Level-3 numerical kernels at 3/3 seeds where C1 and C2g both fail. C4tl solves all four core Level-4 designs at 5/5 seeds, including the 16-point FFT and IFFT. Golden-verified solves extend to Level-5 convolution and image processing and to Level-6 AES encryption and decryption. At the same time, C2g is a strong baseline: it solves several Level-5 rows and multiple repaired-contract designs that decomposed methods do not. The contribution is not a claim that decomposition dominates monolithic repair. It is evidence that verifier-grounded synthesis, in both monolithic and decomposed forms, can cross parts of the reported hard-task frontier, and that benchmark contracts must be audited before either successes or failures are interpreted.

This paper makes three contributions:

(1) The proposed work introduces an executable-contract-aware evaluation discipline for autonomous RTL synthesis: released contracts, minimally repaired executable contracts, and held rows are separated before positive claims are made.
(2) The proposed work gives a verifier-grounded synthesis algorithm that instantiates direct generation, monolithic repair, decomposition-guided repair, and testbench-localized repair as controlled ablations of feedback, decomposition, reference validation, and localization.
(3) The proposed work demonstrates hard ArchXBench solves beyond the reported direct-generation frontier, including robust Level-4 original-contract FFT/IFFT solves and golden-verified Level-5/Level-6 solves.

[1] Repository URL withheld for double-blind review.

2 Related Work

2.1 LLM-Based RTL Generation

VerilogEval introduced an HDLBits-style benchmark for Verilog generation with automatic functional checking [4]. RTLLM proposed an open benchmark for design RTL generation with large language models [5]. Both benchmarks made RTL generation measurable, but most tasks are small instructional circuits rather than deeply structured systems.

More recent work moves beyond direct prompting. AutoChip uses compiler and simulator feedback to improve HDL generation [10]. VerilogCoder introduces graph-based planning and AST-based waveform tracing [1]. AutoVeriFix uses simulation discrepancies to correct LLM-generated Verilog [9]. Spec2RTL-Agent studies LLM-agent systems for specification-to-RTL generation [14]. VFlow searches over agentic workflows using Monte Carlo tree search [13]. HDLxGraph uses HDL-specific graph retrieval to support repository-level HDL tasks [15]. Evolutionary methods (REvolution, EvolVE, COEVO) use inference-time search or co-evolutionary optimization for RTL correctness and power/performance/area [2, 6, 7]. RTL-BenchMT studies agent-assisted benchmark maintenance, including flawed-case revision and overfitting detection [12]. FormalRTL integrates software references and formal equivalence checking into a multi-agent RTL synthesis flow [3]. RTL-BenchLS proposes a large formally verified RTL benchmark for reasoning and generation tasks [11].

Our work evaluates on ArchXBench Levels 3–6, including rows at and above the reported Level-4 frontier. It separates original benchmark contracts from repaired executable contracts. This distinction matters because an RTL benchmark can fail either because the generated design is wrong or because the released testbench, file format, or golden comparator is inconsistent. RTL-BenchMT validates that benchmark ambiguity is a first-order issue; FormalRTL and RTL-BenchLS show the value of stronger executable or formal oracles. The present study uses benchmark-contract audit to support a hard-ArchXBench synthesis study rather than to propose a general benchmark-maintenance system.

Table 1 summarizes the positioning.

2.2 Verifier-Grounded Synthesis

Counterexample-guided inductive synthesis (CEGIS) uses a verifier to reject incorrect candidates and guide the next attempt. LLM-based repair loops instantiate a related pattern: generate code, run tests, inspect errors, and repair. For RTL, verifier feedback is heterogeneous: failures arise from syntax, elaboration, simulation, timing, protocol mismatch, file-output mismatch, or incorrect golden references. The present pipeline treats simulator and golden-output feedback as first-class signals and audits the benchmark contract when executable artifacts disagree.

3 Problem Setting

Each ArchXBench design provides a natural-language specification and executable assets: Verilog testbenches, input files, output files, and comparison scripts. An autonomous synthesis condition receives the task and emits RTL. For self-checking designs, a run succeeds only when the official testbench passes all checks. For file-output designs, simulation completion is not sufficient; the generated output must match the golden output element by element.

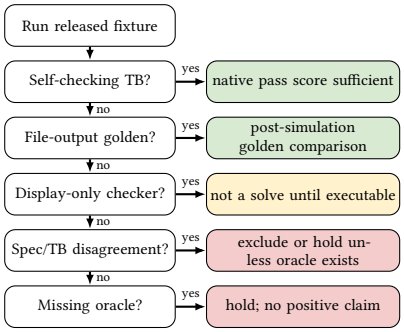

Figure 1: Executable-contract classification. Positive results are claimed only when the released or repaired fixture has an executable oracle.

Three evaluation classes are distinguished:

- **Original-contract results:** runs against the released benchmark as-is.
- **Repaired-contract results:** runs against a copied fixture with minimally repaired infrastructure inconsistencies.
- **Held or excluded rows:** tasks where the released contract remains too ambiguous for a credible positive result.

Figure 1 gives the contract-classification procedure used during the audit.

4 Method

4.1 Evaluated Conditions

Table 2 summarizes the operational differences among the four conditions. All conditions receive the same specification, module interface, and released testbench. The difference is how much executable feedback enters the loop and whether the design is treated as a monolith or decomposed into modules. Thus the four conditions are also ablations: C1 removes feedback, C2g removes decomposition, C4i adds decomposition without localized repair, and C4tl adds a strict reference gate plus localization.

C1: direct generation. The model receives the specification and testbench excerpt and emits RTL five independent times. The candidate with the highest pass count under the executable checker is retained. No repair feedback is used.

C2g: monolithic golden-feedback repair. The model studies the specification, interface, and testbench, then emits a complete top module. Each round compiles and simulates the candidate. Compile errors, self-checking test failures, or golden-output mismatches (with index-level expected-versus-actual values) are appended to the conversation. The model returns a corrected module. The conversation is pruned to the first two and last sixteen messages when it exceeds twenty entries. The loop terminates when all checks pass or after 30 rounds. This is a deliberately strong baseline: it uses the same executable oracle as decomposed methods without paying module-integration risk.

C4i: decomposition-guided iterative repair. C4i asks the model to decompose the task into 3–7 submodules with explicit

Table 1: Positioning relative to recent LLM-aided RTL work. The key distinction is a hard-ArchXBench study combining verifier-grounded synthesis with explicit original-contract versus repaired-contract accounting.

Work	Primary target	Feedback / search	Benchmark style	Difference from this study
VerilogEval / RTLLM	Generation benchmarks	Testbench eval.	HDLBits / moderate RTL	Establishes automatic evaluation; mostly easier than ArchXBench L4–L6
AutoChip / AutoVeriFix	HDL repair	Compiler, simulator, oracle	Repair/generation tasks	Iterative correction is related; no benchmark-contract audit
VerilogCoder / Spec2RTL	Autonomous agents	Planning, tracing, multi-agent	VerilogEval / RTLLM	Strong agents; not focused on ArchXBench hard-level contracts
VFlow	Workflow search	MCTS over workflows	Generation tasks	Searches workflows; does not study contract validity
REvolution / EvolVE / COEVO	Evolutionary search	Population search, PPA	VerilogEval / RTLLM	Search-oriented; does not study benchmark contracts
RTL-BenchMT	Benchmark maintenance	Agent-assisted revision	VerilogEval / RTLLM / CVDP	Validates contract concerns; focused on maintenance
FormalRTL / RTL-BenchLS	Formal verification	Equivalence checking	Formally verified suites	Stronger oracles; not focused on ArchXBench hard rows
This work	Hard RTL synthesis	Golden repair, decomposition, audit	ArchXBench L3–L6	Separates original, repaired, and held rows

Table 2: Operational differences among synthesis conditions. "Ref. gate" indicates whether the decomposition's reference composition must pass the system testbench before synthesis proceeds. "Local repair" describes the granularity of repair targeting.

Cond.	Decomp.	Ref. gate	Golden fb.	Local repair	Budget
C1	no	–	no	–	5 shots
C2g	no	–	yes	whole design	30 rounds
C4i	yes	sanity	yes	per module	≈28 rds
C4tl	yes	strict	yes	culprit only	3+≈28 rds

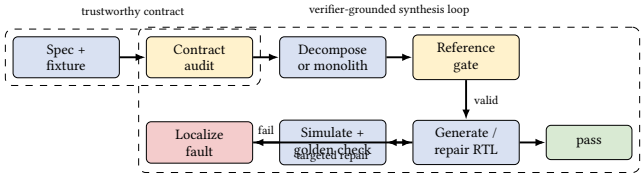

Figure 2: C4tl pipeline. The method first establishes an executable contract, then validates a reference decomposition before entering a golden-feedback loop. On failure, swap-based localization chooses one culprit module for targeted repair.

interfaces, a top-level wrapper, and reference implementations. Submodules are constrained to be combinational where possible; sequential logic is concentrated in the top module. The reference composition (top plus all reference submodules) is simulated against the system testbench as a sanity check.

After decomposition, each submodule is solved in sequence. The model receives the submodule specification, its position in the pipeline relative to neighbors, the reference implementation, and a testbench excerpt. A study prompt requires the model to explain its understanding before writing code. During repair, all unsolved submodules are held at their reference implementations, isolating bugs to the module under repair. Each submodule receives $\lfloor 28/n \rfloor$ repair rounds for n submodules. Golden-output mismatches are fed back with specific failing indices and expected values.

C4tl: testbench-localized decomposed repair. C4tl uses the same decomposition structure as C4i but adds two mechanisms: a strict reference gate and trace-lifted fault localization.

The reference gate requires the composed reference implementation to pass the system-level testbench before synthesis begins. If the reference fails, the decomposition is retried up to three times with failure feedback augmenting the specification. If no valid decomposition is found, the run aborts. This prevents ungrounded decompositions from entering the synthesis loop.

After initial candidate generation for all submodules, C4tl runs the full composed design against the system testbench. On failure, trace-lifted localization identifies the culprit module: each candidate submodule is swapped with its validated reference one at a time while all other candidates remain fixed. The module whose swap produces the largest score improvement is selected for repair. Only that module is repaired using the failing simulator output, golden mismatch details, per-module swap scores, and the reference implementation. The model is instructed to trace the datapath for the first failing input, compute expected intermediate values, identify the root cause, and then fix. This loop repeats until the system checker passes or the budget is exhausted.

The localization mechanism is the central algorithmic distinction. C4i repairs modules in a fixed order; C4tl uses system-level evidence to identify which module is most implicated. For tightly coupled pipelines such as the 16-point FFT, this targeted approach avoids diffuse repair of modules that may not be at fault. The localization step is heuristic rather than a correctness guarantee. It is strongest when the reference composition is valid and one module dominates the observed failure. It can be misled by coupled multi-module bugs, bad decompositions, or errors whose effects are masked until multiple modules are replaced together.

4.2 Artifact and Verification Discipline

Each run is recorded under a repository artifact tree with a structured result file containing the design, condition, model, seed, pass counts, golden counts, LLM-call count, wall time, and generated RTL. File-output designs require strict golden verification: the generated output is compared element by element against the golden reference with a tolerance of ±1 on integer values. Native simulation completion alone is treated as diagnostic evidence, not as a solve.

Table 3: Algorithm 1: executable-contract-aware verifier-grounded RTL synthesis. The condition parameter selects C1, C2g, C4i, or C4tl by enabling feedback, decomposition, reference gating, and localization.

Input: specification S, released fixture B, condition c, seed s, budget T
Output: SOLVED, UNSOLVED, or HELD with artifact record

1 Classify B using Fig. 1. If no executable oracle can be recovered, return HELD.
2 If c uses decomposition, ask the model for interfaces, top wrapper, and reference modules $R_1, \ldots, R_n$; otherwise set $n = 1$ and target the whole design.
3 If c uses a reference gate, simulate the composed references against B; retry decomposition with failure feedback, and abort if no valid reference composition is found.
4 Generate candidate RTL modules $M_1, \ldots, M_n$ from S, B, references, and seed s.
5 For each repair round $t \leq T$: simulate the composed candidate; compute pass/golden score; if all executable checks pass, save RTL and return SOLVED.
6 Select a repair target. C2g selects the whole design; C4i selects the next module in fixed order; C4tl swaps each M_i with R_i and selects the module with the largest system-score improvement.
7 Repair only the selected target using compile errors, failing indices, expected/actual values, swap scores, and the relevant reference implementation; then continue from Step 5.
8 Save the final artifact and return UNSOLVED.

Table 4: Per-design cost for selected solved rows (seed-averaged). Level-4 rows average over 5 seeds; all others over 3.

Design	Method	Avg. calls	Avg. wall
`fp_multiplier` (L3)	C4i	5.0	5.2 min
`gauss_siedel` (L3)	C4i	14.3	11.9 min
`fft_16pt_iterative` (L4)	C4tl	6.0	5.1 min
`ifft_16pt_iterative` (L4)	C4tl	6.2	5.5 min
`harris_corner` (L5)	C4i	10.7	9.4 min
`aes_encryption` (L6)	C4i	7.3	7.1 min

5 Experimental Setup

5.1 Benchmark Scope

The evaluation covers ArchXBench Levels 3–6: iterative arithmetic, FFT/IFFT, floating-point pipelines, FIR filters, image-processing kernels, matrix multiplication, and AES. Lower levels are excluded because the ArchXBench paper reports that current models already solve them. Execution uses repository scripts with Python orchestration, Icarus Verilog (`iverilog`), and the VVP runtime (`vvp`); each candidate is compiled before simulation and self-checking or golden-output comparison.

5.2 Seeds, Model, and Metrics

Main tables use seeds 42, 123, and 456. Two additional seeds (789, 1024) provide robustness evidence for C4tl Level-4 rows. These are fixed seeds chosen for reproducibility, not tuned seeds. C1 uses five seeds for the two borderline Level-3 floating-point rows, `fp_adder` and `fp_multiplier`, to give a stricter direct-generation baseline estimate. The primary model is GPT-5.5 as recorded in the repository artifacts.

A clean solve requires all official checks to pass (self-checking designs) or all golden outputs to match (file-output designs). Solve rates are reported over seeds.

All methods run as executable synthesis loops; the method budgets are summarized in Table 2.

Table 4 reports per-design statistics for selected solved rows, averaged over seeds. These illustrate the practical cost of individual solves: the Level-4 FFT solves in roughly 5 minutes and 6 LLM calls, while harder designs such as `harris_corner_detection` require 10.7 calls and 9.4 minutes.

6 Crossing the Original-Contract Frontier

Table 5 reports the main original-contract results.

Level 3. C4i provides the strongest method-value evidence: it solves five numerical kernels at 3/3 seeds where C1 scores 0/3–0/5 and C2g scores 0/3–1/5. C4tl matches C4i on four of five rows but solves `gauss_siedel` at only 1/3. For context, C4i decomposes `gauss_siedel` into three submodules (reciprocal unit, iteration step, solution selector), allowing each to be solved and tested in isolation. The negative result on `newton_raphson_polynomial` is an original-checker ceiling: three of its 100 assertions are structurally unsatisfiable, capping the achievable score at 97/100. After removing only those three unsatisfiable residual assertions in the repaired-contract fixture, C2g solves all three seeds (Table 7), confirming that the design itself is within model capability.

Level 4. C4tl solves all four core Level-4 rows at 3/3 main seeds and 2/2 robustness seeds, for 5/5 total. The 16-point FFT and IFFT rows are the strongest hard-frontier evidence because they are self-checking original-contract solves with no benchmark repair. C4tl decomposes `fft_16pt_iterative` into four submodules (bit-reverse mapper, twiddle ROM, butterfly unit, output scaler) and uses trace-lifted localization to identify which module is at fault after each simulation failure. C2g also solves the FFT/IFFT rows at 3/3; C4i does not. C4i's fixed sequential repair order can spend iterations on modules that already compose correctly while the actual fault remains in a later module; C4tl's swap-based localization directly identifies the culprit before repair. The pipeline rows (`fp_adder_pipeline`, `fp_mult_pipeline`) are solved by all conditions.

Levels 5–6. C4i obtains 3/3 golden-verified solves on `conv1d`, `harris_corner_detection`, and both AES rows. C4i decomposes AES encryption into six submodules (S-box, SubBytes, ShiftRows, MixColumns, round function, key expansion), each implemented and tested against the system testbench with other modules held at their references. Several Level-5 rows are solved only by C2g: `conv2d` (4,096/4,096), `dct_idct_8pt_pipelined` (16/16), and `unsharp_mask` (65,536/65,536). These rows demonstrate that monolithic repair is the stronger strategy when the design is compact enough for whole-design feedback or when the primary difficulty is a global convention such as file format or fixed-point encoding.

The results are not presented as a universal decomposition win. C2g is a serious baseline: it matches or outperforms decomposition on 8 of 17 original-contract rows. The most accurate interpretation is that decomposition provides value on specific hard rows and gives a structured synthesis process, while monolithic golden-feedback repair is often sufficient and sometimes better.

Table 5: Original-contract result heatmap on ArchXBench. Green cells solve all main seeds; yellow cells are partial; red cells fail. GV marks golden-verified file-output rows. Level-4 C4tl rows additionally solve robustness seeds 789 and 1024, giving 5/5 total.

Level	Design	C1	C2g	C4i	C4tl	Evidence
L3	fp_adder	0/5	1/5	**3/3**	**3/3**	decomposition wins
L3	fp_multiplier	0/5	0/3	**3/3**	**3/3**	decomposition wins
L3	gauss_siedel	0/3	0/3	**3/3**	1/3	C4i strongest
L3	gradient_descent	0/3	0/3	**3/3**	**3/3**	decomposition wins
L3	newton_raphson_sqrt	0/3	0/3	**3/3**	**3/3**	decomposition wins
L3	newton_raphson_poly.	0/3	0/3	0/3	0/3	checker ceiling at 97/100
L4	fft_16pt_iterative	0/3	**3/3**	0/3	**3/3**	hard-row solve; 5/5
L4	ifft_16pt_iterative	0/3	**3/3**	0/3	**3/3**	hard-row solve; 5/5
L4	fp_adder_pipeline	**3/3**	**3/3**	**3/3**	**3/3**	all methods solve; 5/5
L4	fp_mult_pipeline	**3/3**	**3/3**	**3/3**	**3/3**	all methods solve; 5/5
L5	conv1d	**3/3**	**3/3**	**3/3**	1/3	GV
L5	conv2d	0/3	**3/3**	0/3	0/3	GV; C2g strongest
L5	dct_idct_8pt_pipelined	0/3	**3/3**	0/3	0/3	GV; C2g strongest
L5	harris_corner_detection	0/3	**3/3**	**3/3**	0/3	GV
L5	unsharp_mask	0/3	**3/3**	0/3	0/3	GV; C2g strongest
L6	aes_encryption	**3/3**	**3/3**	**3/3**	2/3	GV
L6	aes_decryption	1/3	**3/3**	**3/3**	1/3	GV; repair improves C1

newton_raphson_poly. abbreviates newton_raphson_polynomial.

Table 6: Second-model validation with Claude Sonnet 5 on selected hard-frontier rows. GV denotes golden-verified file-output rows.

Design	C2g	C4tl
fft_16pt_iterative	**3/3**	**3/3**
ifft_16pt_iterative	**3/3**	**3/3**
aes_encryption	**3/3** GV	0/3 GV
aes_decryption	**3/3** GV	0/3 GV

Table 7: Repaired-contract results, separated from original ArchXBench claims. These distinguish benchmark-contract failures from model capability failures.

Design	C2g	C4i	C4tl
conv_3d	3/3	2/3	0/3
multich_conv2d	3/3	3/3	3/3
quantized_matmul	3/3	3/3	0/3
fp_band_pass_fir	3/3	0/1	0/1
fp_high_pass_fir	3/3	0/1	0/1
newton_raphson_poly.	3/3	1/3	1/3
systolic_gemm	0/3	0/3	0/3

Model generality. To test whether the hard-frontier result is specific to gpt-5.5, Table 6 repeats the strongest original-contract rows with Claude Sonnet 5. The original Level-4 FFT/IFFT rows solve on all three seeds under both C2g and C4tl, and the Level-6 AES rows solve on all three seeds under C2g with golden verification. Sonnet 5 does not solve AES under C4tl because the reference decomposition fails before localized repair can begin. For AES encryption, Sonnet 5 produces decompositions whose reference compositions fail the original system testbench; for AES decryption, it does not produce a usable reference composition within the generation budget. This indicates model-dependent sensitivity in the decomposition gate, while monolithic golden-feedback repair remains robust on the same AES rows.

Case study: original Level-4 FFT. fft_16pt_iterative is a self-checking original-contract row, so no benchmark repair is involved. C1 fails on all three main seeds, while C2g and C4tl solve all three. The row therefore isolates the central mechanism: executable feedback can turn a direct-generation failure into a verified RTL solve without changing the released checker.

7 Executable-Contract Audit

Hard RTL benchmarks are useful only when their executable contracts reflect the intended task. During evaluation, several rows were found to contain inconsistent specifications, stale comparators, display-only checkers, or output-schema mismatches. The released benchmark is not overwritten. Repaired fixtures are kept under a separate artifact root and reported separately.

The audit follows a fixed protocol. First, the released fixture is run unchanged. Second, the checker is classified (self-checking, file-output with golden, display-only, stale, specification-inconsistent, or missing-oracle). Third, file-output rows are judged only by post-simulation golden comparison. Fourth, infrastructure repairs are applied only in copied fixtures and only when the intended oracle is recoverable from released assets. Fifth, unresolved rows are held rather than converted into positive results. The repair rule is intentionally narrow: a repaired fixture may make an existing oracle executable or repair file-format infrastructure, but it may not change the semantic task, introduce hidden target behavior, or move a row into the original-contract table.

Table 7 reports the repaired-contract evidence. These rows are not counted as original ArchXBench solves.

The audit produces both positive and negative evidence. Repairing the executable contract for multich_conv2d and quantized_matmul makes the intended task measurable and solvable. By contrast, converting the display-only systolic_gemm checker into executable checks does not produce a win: all methods remain at 0/3. This is a genuine remaining capability boundary, not a benchmark-repair success.

Several rows remain held or excluded. The L4 FIR family is excluded because the specification and testbench disagree on filter coefficients. The L6 `fp_low_pass_fir` is held because the released files do not expose an explicit coefficient oracle. The L6 `fft_streaming_64pt` is excluded because the released input/output contract has unresolved schema and numeric-encoding ambiguities.

8 Discussion

8.1 When Decomposition Helps and When It Does Not

Decomposition is most effective when a design factors naturally into modules and when failure localization reduces the repair burden. The Level-3 numerical kernels and Level-4 FFT/IFFT rows illustrate this: C4i and C4tl solve rows that both C1 and C2g fail on. C4i fails on the FFT/IFFT rows where C4tl succeeds. One explanation is that C4i's fixed per-module repair order is less effective for tightly coupled pipeline stages; C4tl's trace-lifted localization identifies the culprit module and avoids diffuse iteration on modules that are not at fault.

By contrast, C2g solves several Level-5 rows that neither C4i nor C4tl can solve. This happens when the design is compact enough for whole-design repair, when module boundaries introduce integration risk, or when the primary difficulty is a global convention such as file format or fixed-point encoding. The paper's claim is therefore conservative: decomposition is a powerful synthesis strategy for some hard rows, but monolithic repair remains strong and should be treated as a primary baseline.

8.2 Why Benchmark Contracts Matter

LLM-aided RTL synthesis papers increasingly rely on executable benchmarks. If the executable contract is ambiguous, a model can fail for the wrong reason or pass the wrong task; separating original-contract and repaired-contract results is therefore a prerequisite for credible evaluation.

9 Limitations

The evaluated methods do not establish universal dominance over monolithic repair; C2g should be treated as a primary baseline. Some historical rows are trusted score-only rather than artifact-backed and are labeled accordingly. Repaired-contract evaluation depends on judgment about what constitutes an infrastructure repair rather than a semantic benchmark change; this risk is mitigated by keeping repaired fixtures separate and excluding unresolved rows. The empirical study is concentrated on RTL synthesis; transfer to other executable synthesis domains is not established here.

10 Conclusion

This study evaluated autonomous RTL synthesis on hard ArchXBench designs using iterative repair, modular decomposition, and strict simulator/golden feedback. Original-contract solves span Levels 3–6, including robust Level-4 FFT/IFFT rows and golden-verified Level-5/Level-6 designs. Monolithic golden-feedback repair is a strong baseline. Decomposition is powerful but not universally superior. Benchmark-contract defects can obscure both progress and failure.

Hard autonomous RTL synthesis requires structured generation, executable feedback, and trustworthy benchmark contracts together. Without all three, both successes and failures can be misinterpreted.

References

[1] Chia-Tung Ho, Haoxing Ren, and Brucek Khailany. 2025. VerilogCoder: Autonomous Verilog Coding Agents with Graph-based Planning and Abstract Syntax Tree (AST)-based Waveform Tracing Tool. In *Proceedings of the AAAI Conference on Artificial Intelligence.*

[2] Wei-Po Hsin, Ren-Hao Deng, Yao-Ting Hsieh, En-Ming Huang, and Shih-Hao Hung. 2026. EvolVE: Evolutionary Search for LLM-based Verilog Generation and Optimization. *arXiv preprint arXiv:2601.18067* (2026).

[3] Kezhi Li, Min Li, Xiangyu Wen, Shibo Zhao, Jieying Wu, Junhua Huang, and Qiang Xu. 2026. FormalRTL: Verified RTL Synthesis at Scale. *arXiv preprint arXiv:2603.08738* (2026).

[4] Mingjie Liu, Nathaniel Pinckney, Brucek Khailany, and Haoxing Ren. 2023. VerilogEval: Evaluating Large Language Models for Verilog Code Generation. In *Proceedings of the IEEE/ACM International Conference on Computer-Aided Design.*

[5] Yao Lu, Shang Liu, Qijun Zhang, and Zhiyao Xie. 2023. RTLLM: An Open-Source Benchmark for Design RTL Generation with Large Language Model. *arXiv preprint arXiv:2308.05345* (2023).

[6] Kyungjun Min, Kyumin Cho, Junhwan Jang, and Seokhyeong Kang. 2026. REvolution: An Evolutionary Framework for RTL Generation Driven by Large Language Models. In *Proceedings of the Asia and South Pacific Design Automation Conference.*

[7] Heng Ping, Peiyu Zhang, Shixuan Li, Wei Yang, Anzhe Cheng, Shukai Duan, Xiaole Zhang, and Paul Bogdan. 2026. COEVO: Co-Evolutionary Framework for Joint Functional Correctness and PPA Optimization in LLM-Based RTL Generation. *arXiv preprint arXiv:2604.15001* (2026).

[8] Suresh Purini, Siddhant Garg, Mudit Gaur, Sankalp Bhat, Sohan Mupparapu, and Arun Ravindran. 2025. ArchXBench: A Complex Digital Systems Benchmark Suite for LLM Driven RTL Synthesis. *arXiv preprint arXiv:2508.06047* (2025).

[9] Yan Tan, Xiangchen Meng, Zijun Jiang, and Yangdi Lyu. 2026. AutoVeriFix: Automatically Correcting Errors and Enhancing Functional Correctness in LLM-Generated Verilog Code. In *Proceedings of the Asia and South Pacific Design Automation Conference.*

[10] Shailja Thakur, Jason Blocklove, Hammond Pearce, Benjamin Tan, Siddharth Garg, and Ramesh Karri. 2024. AutoChip: Automating HDL Generation Using LLM Feedback. In *Proceedings of the ACM/IEEE Design Automation Conference.*

[11] Jing Wang, Shang Liu, Wenji Fang, Yuchao Wu, Yugao Zhu, and Zhiyao Xie. 2026. RTL-BenchLS: A Large-Scale Benchmark for RTL Reasoning and Generation with Large Language Models. *arXiv preprint arXiv:2606.08976* (2026).

[12] Jing Wang, Shang Liu, Hangan Zhou, and Zhiyao Xie. 2026. RTL-BenchMT: Dynamic Maintenance of RTL Generation Benchmark Through Agent-Assisted Analysis and Revision. In *Proceedings of the ACM/IEEE Design Automation Conference.* doi:10.1145/3770743.3804053

[13] Yangbo Wei, Zhen Huang, Huang Li, Wei W. Xing, Ting-Jung Lin, and Lei He. 2026. VFlow: Discovering Optimal Agentic Workflows for Verilog Generation. In *Proceedings of the Asia and South Pacific Design Automation Conference.*

[14] Zhongzhi Yu, Mingjie Liu, Michael Zimmer, Yingyan Celine Lin, Yong Liu, and Haoxing Ren. 2025. Spec2RTL-Agent: Automated Hardware Code Generation from Complex Specifications Using LLM Agent Systems. *arXiv preprint arXiv:2506.13905* (2025).

[15] Pingqing Zheng, Jiayin Qin, Fuqi Zhang, Shang Wu, Yu Cao, Caiwen Ding, and Yang Zhao. 2025. HDLxGraph: Bridging Large Language Models and HDL Repositories via HDL Graph Databases. *arXiv preprint arXiv:2505.15701* (2025).